

Aula 08: Aula/Tutorial UVM part-1

Agenda

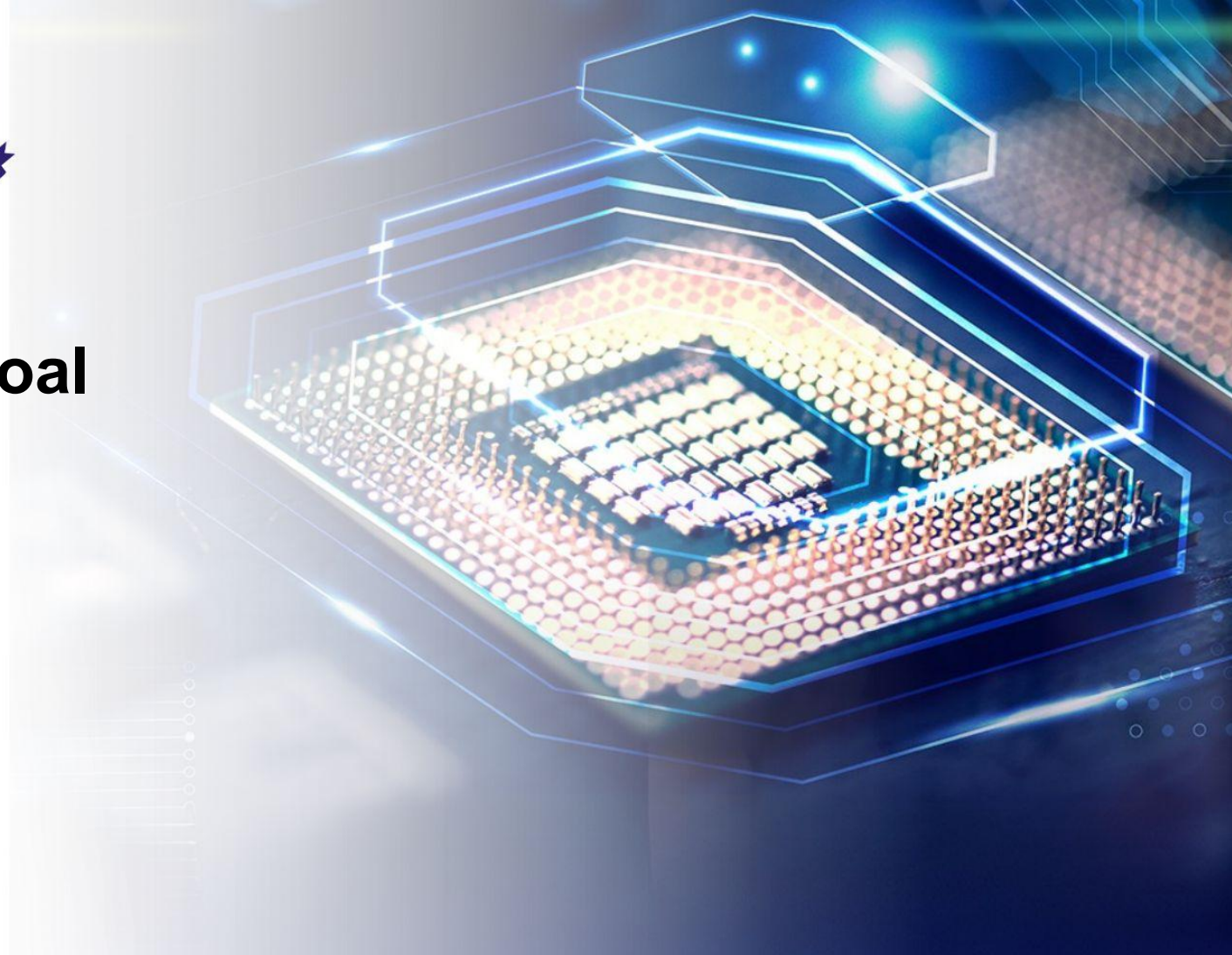
- **Review DV Eng Goal**
- **Why UVM?**
- **How the UVM works?**
 - **POO, Classes, Factory, Phases, ...**
 - **LAB00: "Hello, World!" from test class**
- **UVM More Components with LABs**
 - **... (part2, ...)**

Agenda

- **Part-2**
- UVM TLM: Concepts and Tutorial
 - Producer-Consumer LAB
- Interface / Virtual Interface
- A bit more about Factory
- VIP
- RAL
- UVM phasing jump



Review DV Eng Goal



RESIDÊNCIA EM MICROELETRÔNICA - POLO UFCCG

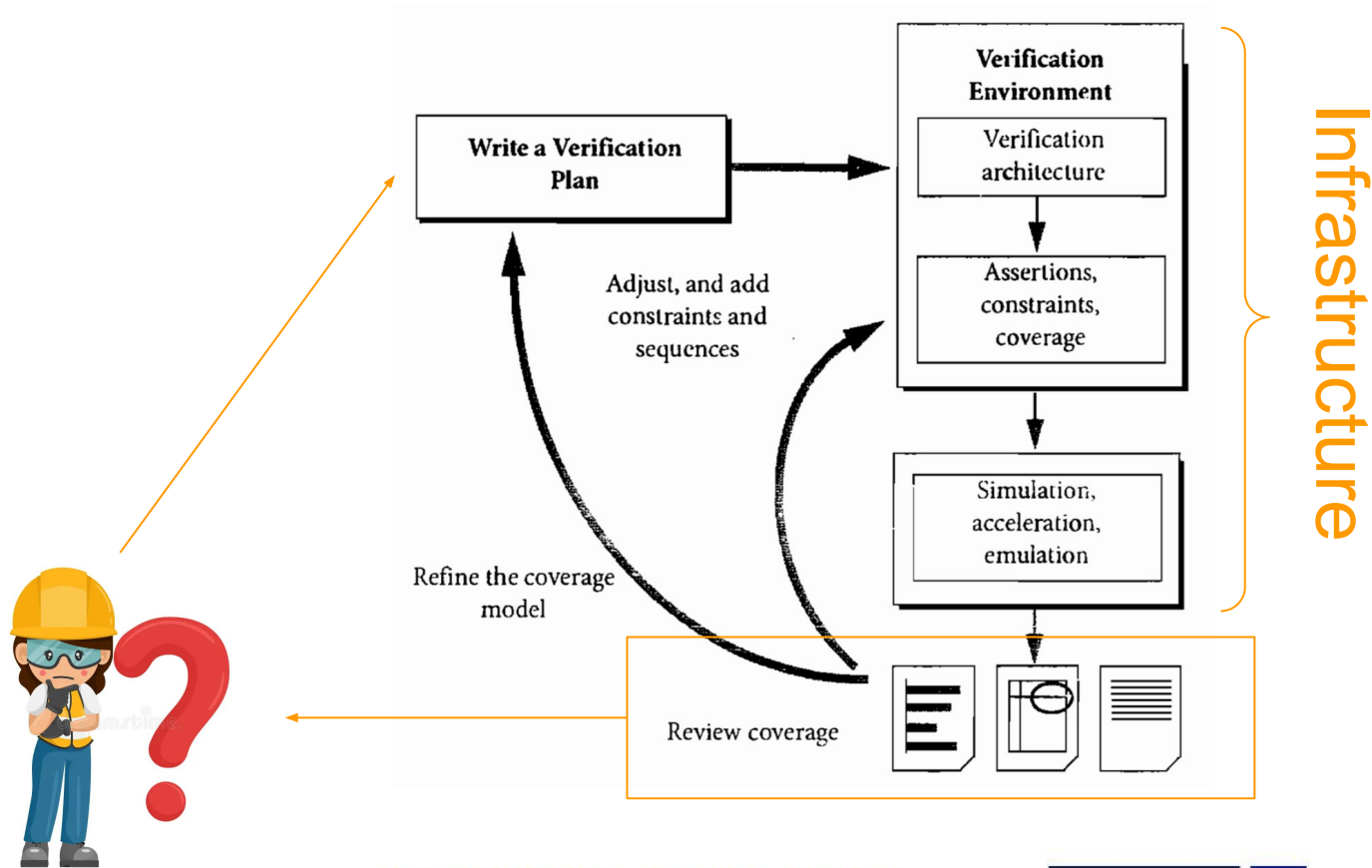


Review DV Eng Goal

- **What is the DV Engineer goal?**
 - Especificar um plano de teste; isso requer estudo do dispositivo, análise dos casos FUNCIONAIS mais importantes
 - Implementar uma infraestrutura de teste escalável;
 - Rodar testes, analisar resultados, refatorar ambiente, reportar bug, até acabar.

Review DV Eng Goal

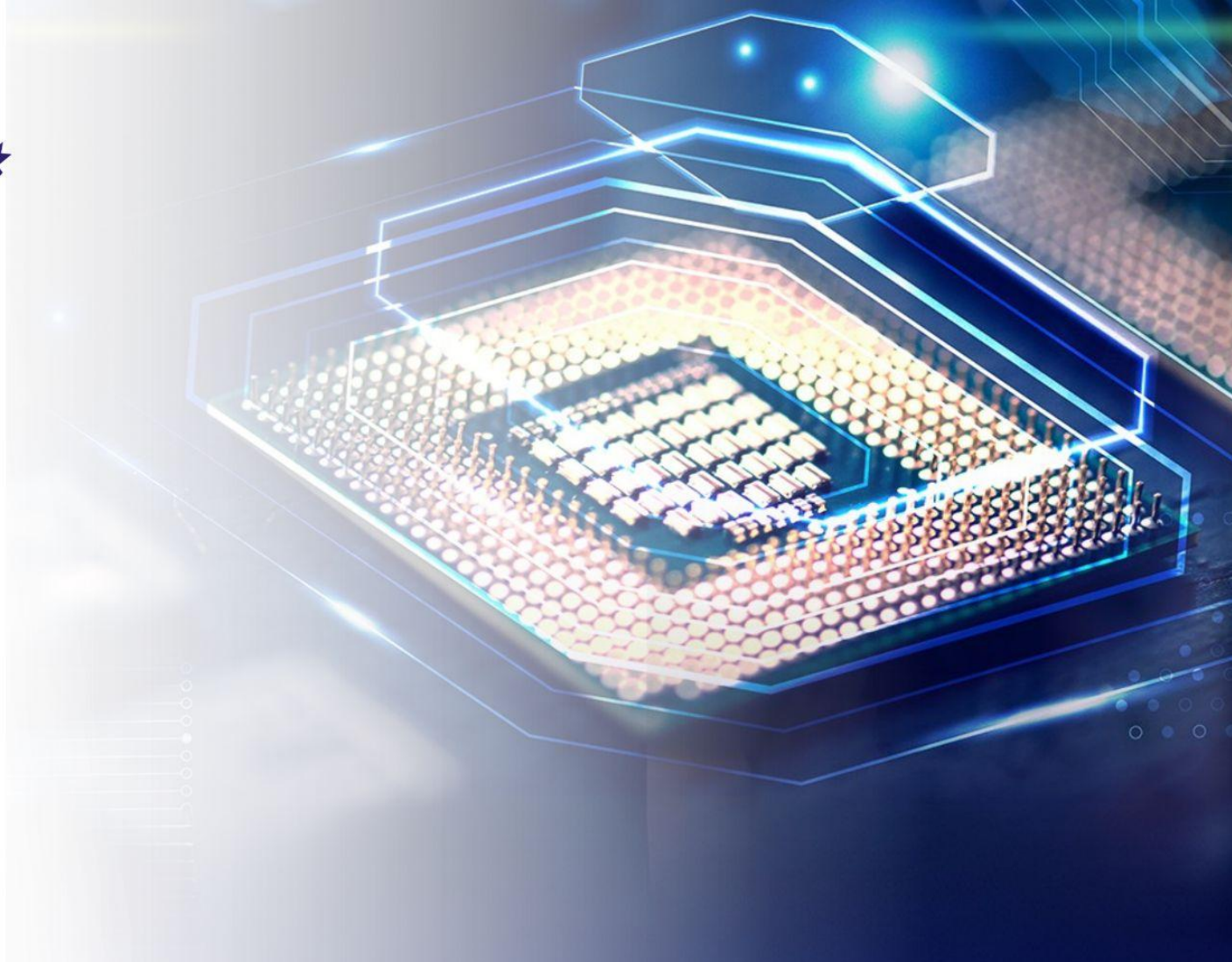
- How?



Why

UVM

?

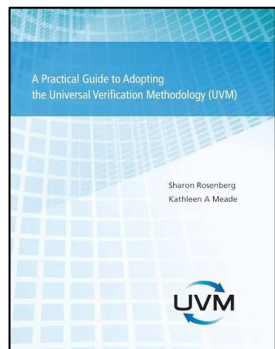


Why UVM?

The Universal Verification Methodology (UVM) is a complete methodology that codifies the best practices for efficient and exhaustive verification. One of the key principles of UVM is to develop and leverage reusable verification components—also called UVM Verification Components (UVCs). The UVM is targeted to verify small designs and large-gate-count, IP-based system-on-chip (SoC) designs.

The UVM has all of the verification features and capabilities your management may ask for to justify its use.

- It is **mature**—The UVM code is based on the Open Verification Methodology (OVM) library, with some modifications made on top of the proven OVM code.
- It is **open**—It is an emerging Accellera standard (and targeted for the IEEE) delivered in an open-source format.
- It is **compatible** and **portable**—It is tested to work on all major commercial simulators.



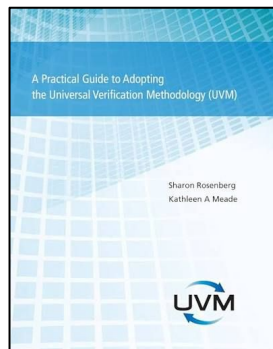
Why UVM?

Metodologia é o estudo ou o conjunto de regras, métodos e processos escolhidos para realizar uma pesquisa, **um projeto** ou uma tarefa. A palavra vem do grego *métodos* (caminho) + *logia* (estudo). Ela define o **passo a passo e a lógica de como** algo **será feito** para atingir um objetivo.

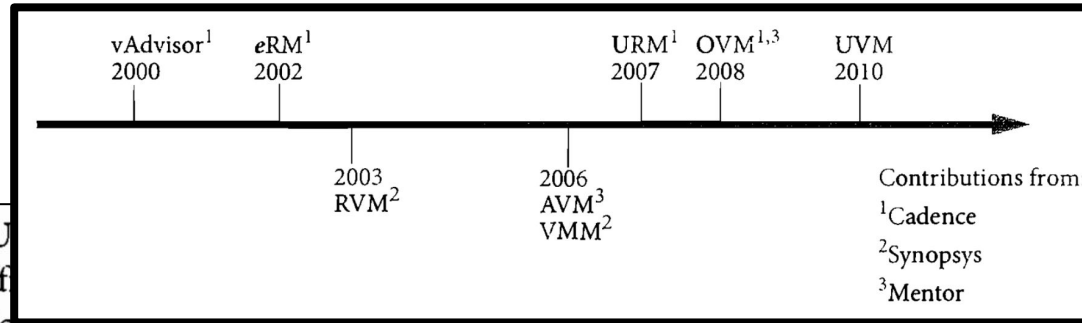
The Universal Verification Methodology (UVM) is a **complete methodology** that codifies the best practices for efficient and exhaustive verification. One of the key principles of UVM is to develop and leverage reusable verification components—also called UVM Verification Components (UVCs). The UVM is targeted to verify small designs and large-gate-count, IP-based system-on-chip (SoC) designs.

The UVM has all of the verification features and capabilities your management may ask for to justify its use.

- It is **mature**—The UVM code is based on the Open Verification Methodology (OVM) library, with some modifications made on top of the proven OVM code.
- It is **open**—It is an emerging Accellera standard (and targeted for the IEEE) delivered in an open-source format.
- It is **compatible** and **portable**—It is tested to work on all major commercial simulators.



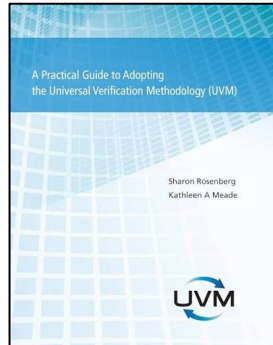
Why UVM?



The UVM is a methodology for efficient verification components—also called UVM Verification Components (UVCs). The UVM is targeted to verify small designs and large-gate-count, IP-based system-on-chip (SoC) designs.

The UVM has all of the verification features and capabilities your management may ask for to justify its use.

- It is **mature**—The UVM code is based on the Open Verification Methodology (OVM) library, with some modifications made on top of the proven OVM code.
- It is **open**—It is an emerging Accellera standard (and targeted for the IEEE) delivered in an open-source format.
- It is **compatible** and **portable**—It is tested to work on all major commercial simulators.



Why UVM?

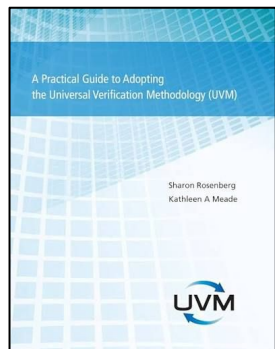
<https://www.accellera.org/downloads/standards/uvm>



The Universal Verification Methodology (UVM) is a complete methodology that codifies the best practices for efficient and exhaustive verification. One of the key principles of UVM is to develop and leverage reusable verification components—also called UVM Verification Components (UVCs). The UVM is targeted to verify small designs and large-gate-count, IP-based system-on-chip (SoC) designs.

The UVM has all of the verification features and capabilities your management may ask for to justify its use.

- It is **mature**—The UVM code is based on the Open Verification Methodology (OVM) library, with some modifications made on top of the proven OVM code.
- It is **open**—It is an emerging Accellera standard (and targeted for the IEEE) delivered in an open-source format.
- It is **compatible** and **portable**—It is tested to work on all major commercial simulators.





VERILATOR

Open Source, but limited

<https://github.com/verilator/uvm>

<https://github.com/chipsalliance/uvm-verilator>

Why UVM?

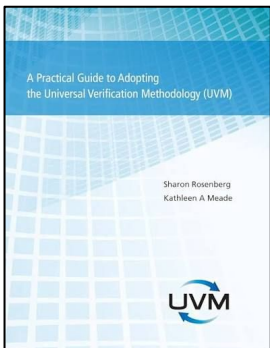


SYNOPSYS[®]
cadence
SIEMENS

The Universal Verification Methodology (UVM) is a complete methodology that codifies the best practices for efficient and exhaustive verification. One of the key principles of UVM is to develop and leverage reusable verification components—also called UVM Verification Components (UVCs). The UVM is targeted to verify small designs and large-gate-count, IP-based system-on-chip (SoC) designs.

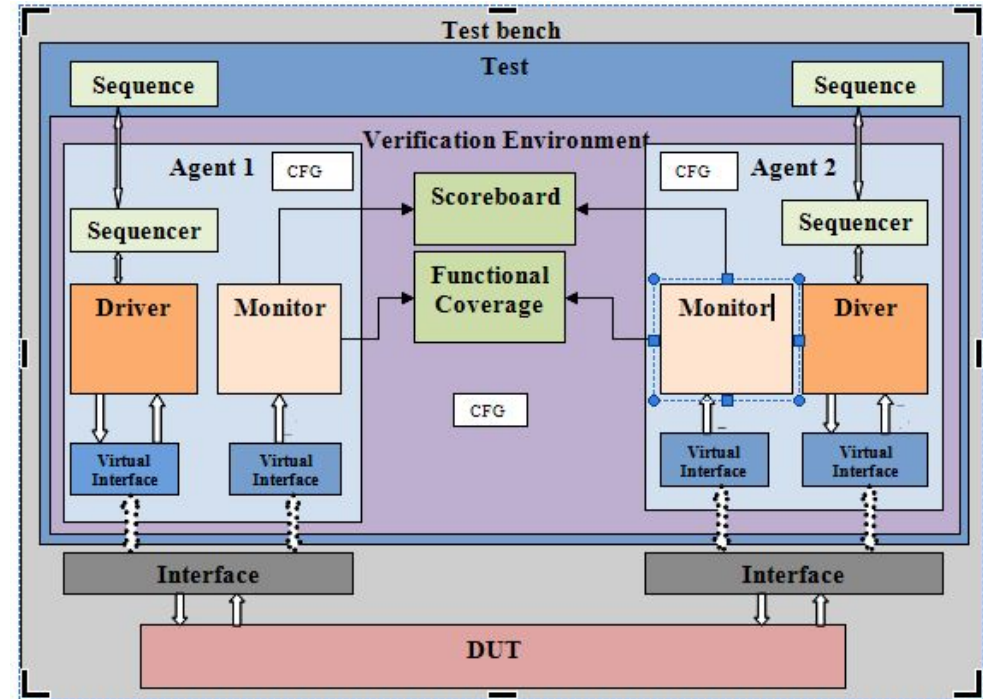
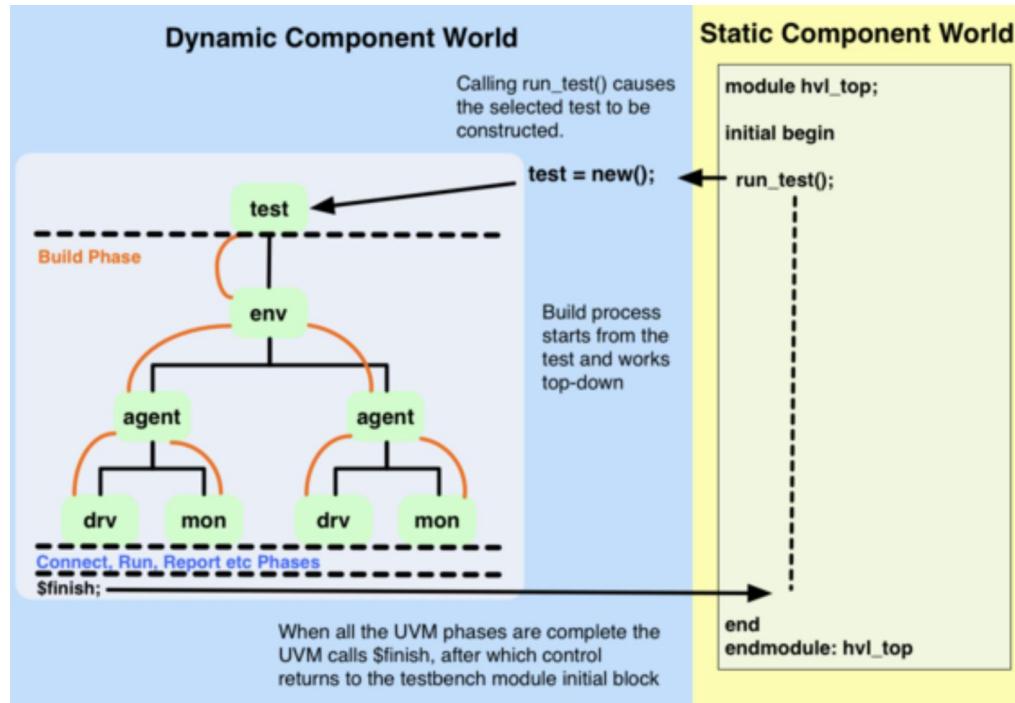
The UVM has all of the verification features and capabilities your management may ask for to justify its use.

- It is **mature**—The UVM code is based on the Open Verification Methodology (OVM) library, with some modifications made on top of the proven OVM code.
- It is **open**—It is an emerging Accellera standard (and targeted for the IEEE) delivered in an open-source format.
- It is **compatible** and **portable**—It is tested to work on all major commercial simulators.



Why UVM?

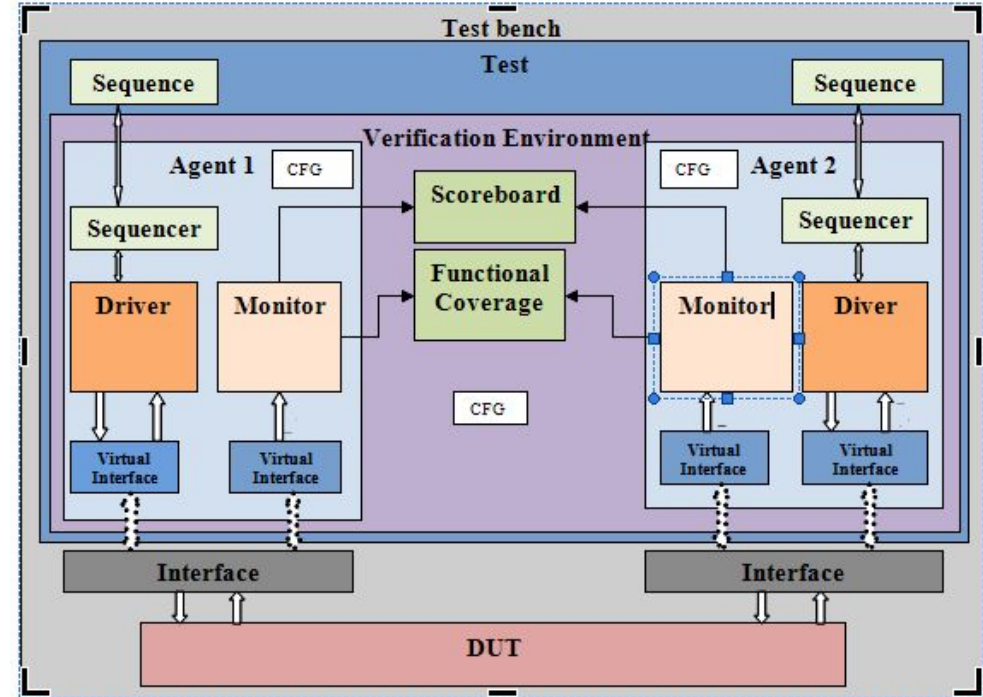
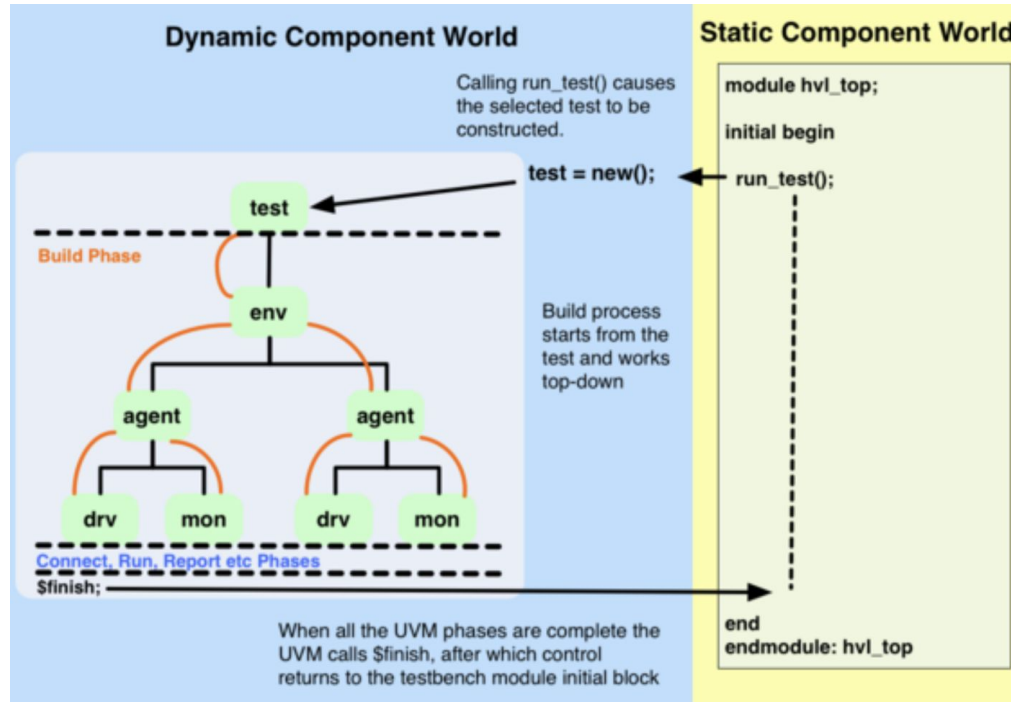
I don't need to think about **HOW TO STRUCTURE THE TEST**; I need to think about **WHAT TO TEST**.



Source: <https://verificationacademy.com/cookbook/uvm-universal-verification-methodology/testbench-build/>

Why UVM?

- The METHOD is done, just USE/CUSTOMIZE it!



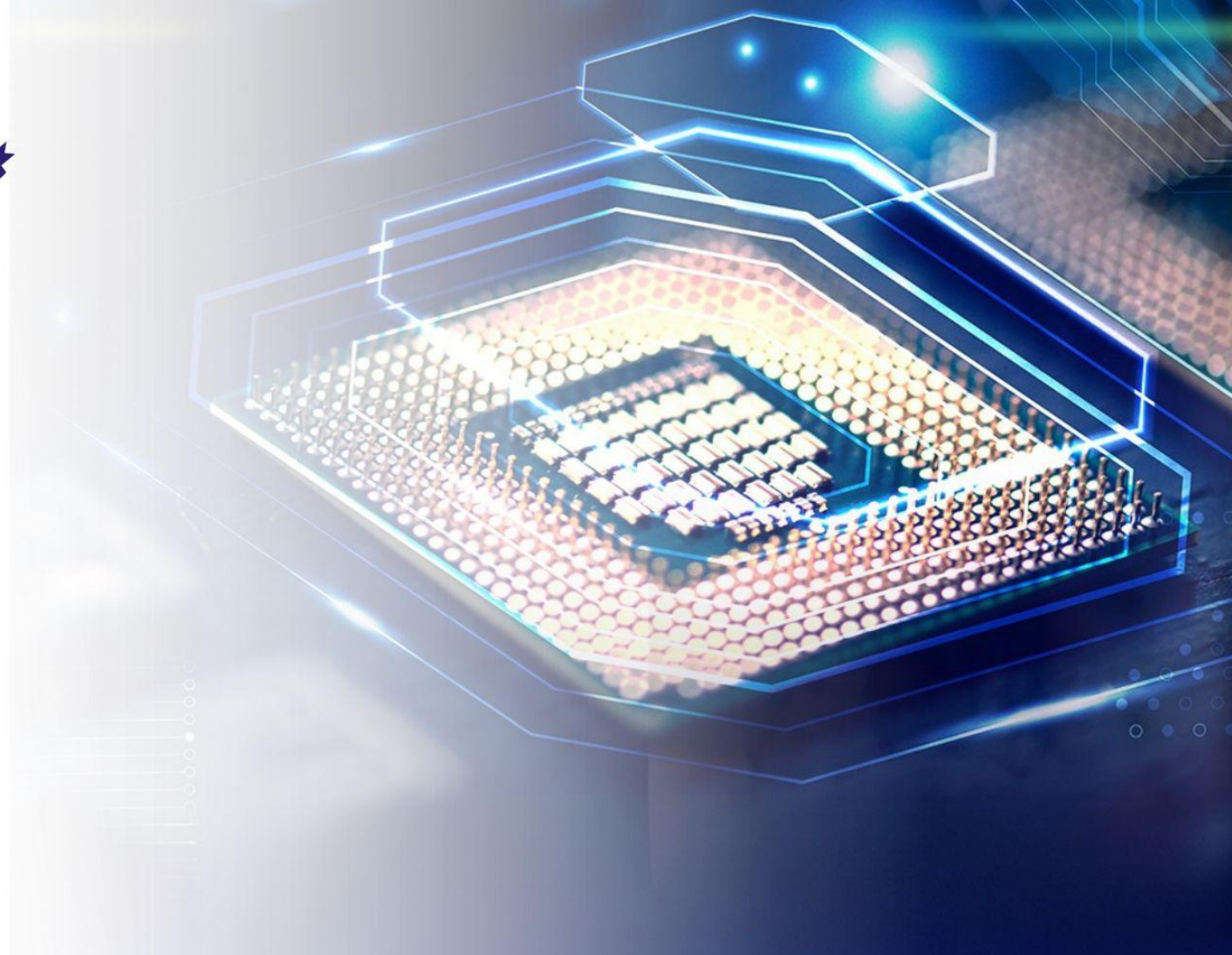
Source: <https://verificationacademy.com/cookbook/uvm-universal-verification-methodology/testbench-build/>



How



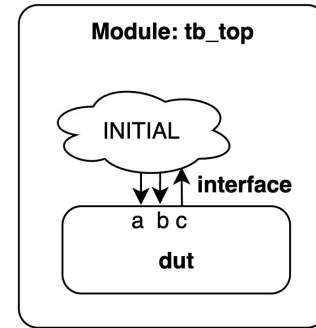
works?



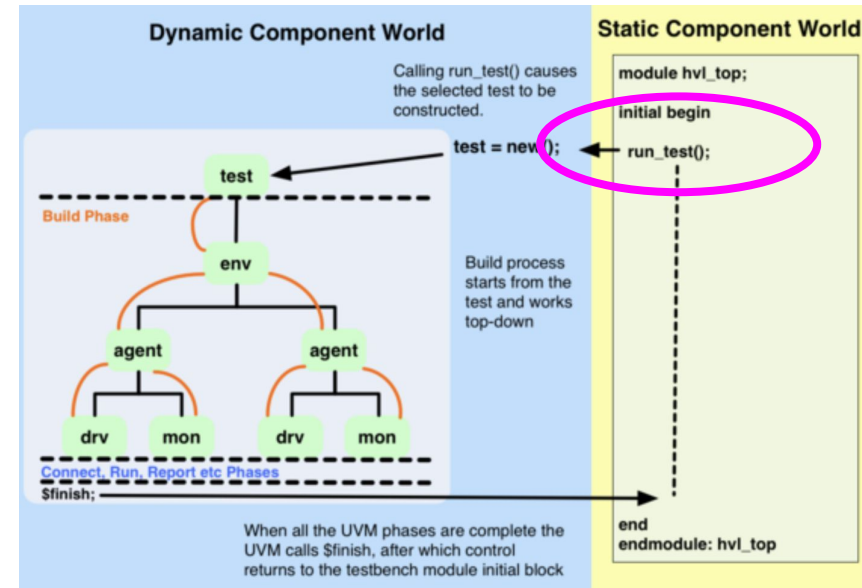
RESIDÊNCIA EM MICROELETRÔNICA - POLO UFMG

How UVM?

- POO behind it:

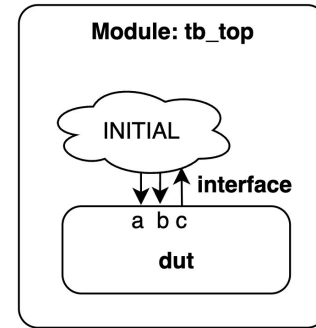


The `run_test()` is the entry point of a POO program.



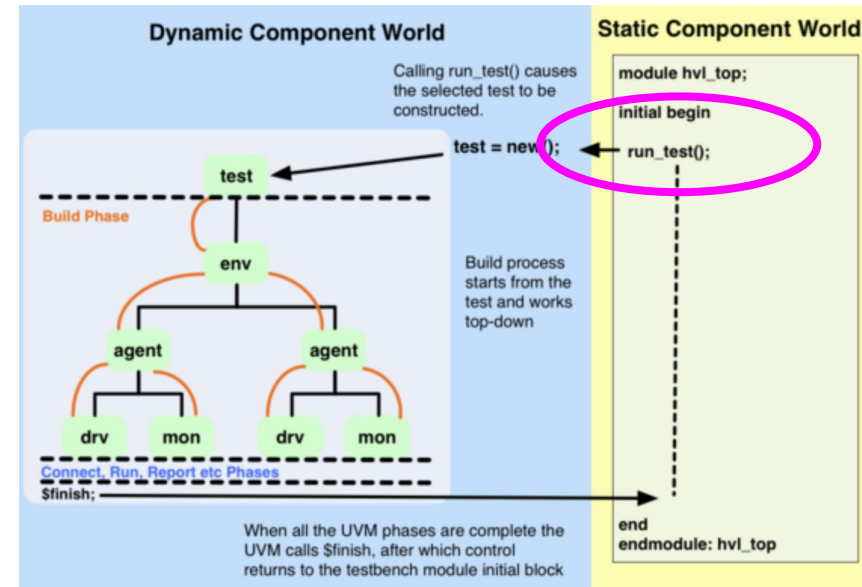
How UVM?

- POO behind it:



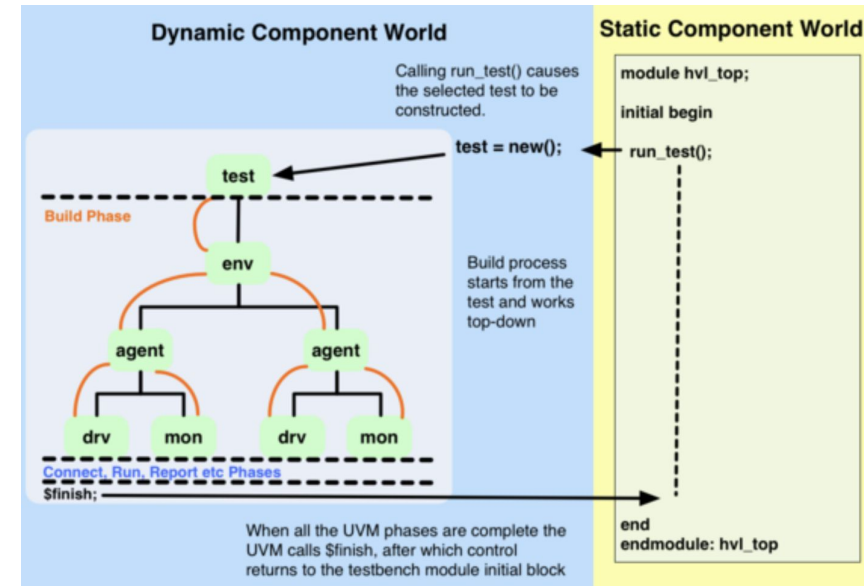
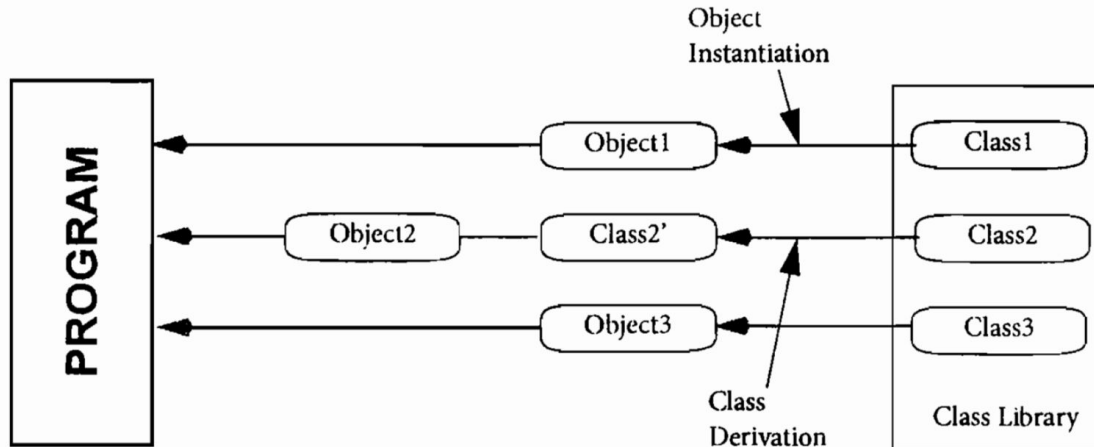
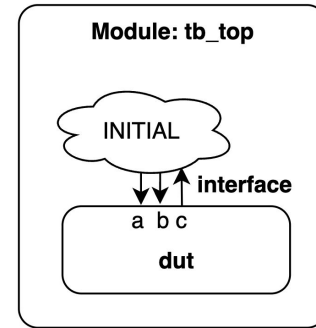
The `run_test()` is the entry point of a POO program.

But, what is a POO program?



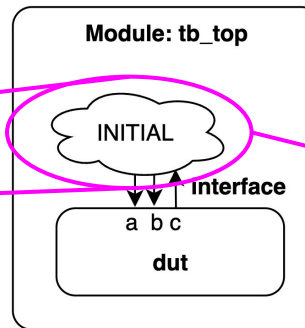
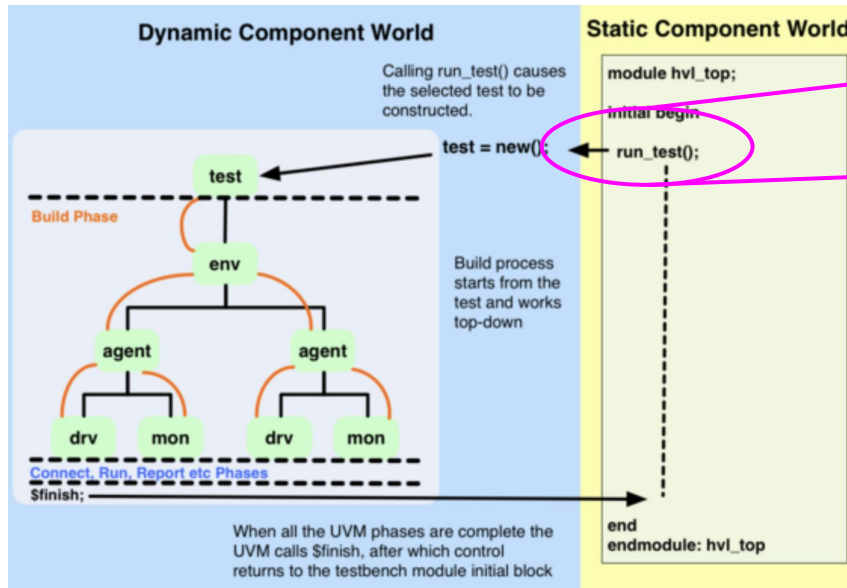
How UVM?

- A **POO** program **ORCHESTRATES** objects.
- **Classes** are handled, instantiated and become objects
- **Classes** are a data structure (modeled)
- **Classes** can be derived (polymorphism)



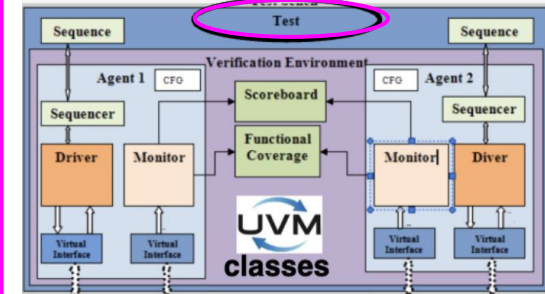
How UVM?

In UVM, the POO program is ready



POO
program

UVM is a collection of classes, interfaces, coverage, etc. i.e., SystemVerilog constructors.



Let's analyze the run_test in the UVM library

```
481 // run_test
482 // -----
483
484 task uvm_root::run_test(string test_name="");
485     uvm_report_server l_rs;
486
487     uvm_factory factory;
488     bit testname_plusarg;
489     int test_name_count;
490     string test_names[];
491     string msg;
492     uvm_component uvm_test_top;
493
494 process phase_runner_proc; // store thread forked below for final cleanup
495
496     uvm_run_test_callback::m_do_pre_run_test();
497
498 factory=uvm_factory::get();
499     m_uvm_core_state.push_front(UVM_CORE_PRE_RUN);
500
501 testname_plusarg = 0;
502
503 // Set up the process that decouples the thread that drops objections from
504 // the process that processes drop/all_dropped objections. Thus, if the
505 // original calling thread (the "dropper") gets killed, it does not affect
506 // drain-time and propagation of the drop up the hierarchy.
507 // Needs to be done in run_test since it needs to be in an
508 // initial block to fork a process.
509 uvm_objection::m_init_objections();
510
511 // Do the same for uvm_process_guard
512 uvm_process_guard_base::m_init_process_guards();
513
514
515 ifndef UVM_NO_DPI
516
517 // Retrieve the test names provided on the command line. Command line
518 // overrides the argument.
519 test_name_count = clp.get_arg_values("+UVM_TESTNAME=", test_names);
520
521 // If at least one, use first in queue.
522 if (test_name_count > 0) begin
```

```
typedef class uvm_default_report_server;
typedef class uvm_cmdline_verbosity;

// Class: uvm_root
//
// Implementation of the uvm_root class, as defined in
// 1800.2-2020 Section F.7 with the following additional API
//
class uvm_root extends uvm_component;

// Function -- NODOCS -- get()
// Static accessor for <uvm_root>.
//
// The static accessor is provided as a convenience wrapper
// around retrieving the root via the <uvm_coreservice_t::get_root>
// method.
//
// | // Using the uvm_coreservice_t:
// | uvm_coreservice_t cs;
// | uvm_root r;
// | cs = uvm_coreservice_t::get();
// | r = cs.get_root();
// |
// | // Not using the uvm_coreservice_t:
// | uvm_root r;
// | r = uvm_root::get();
//
extern static function uvm_root get();

uvm_cmdline_processor clp;

virtual function string get_type_name();
    return "uvm_root";
endfunction

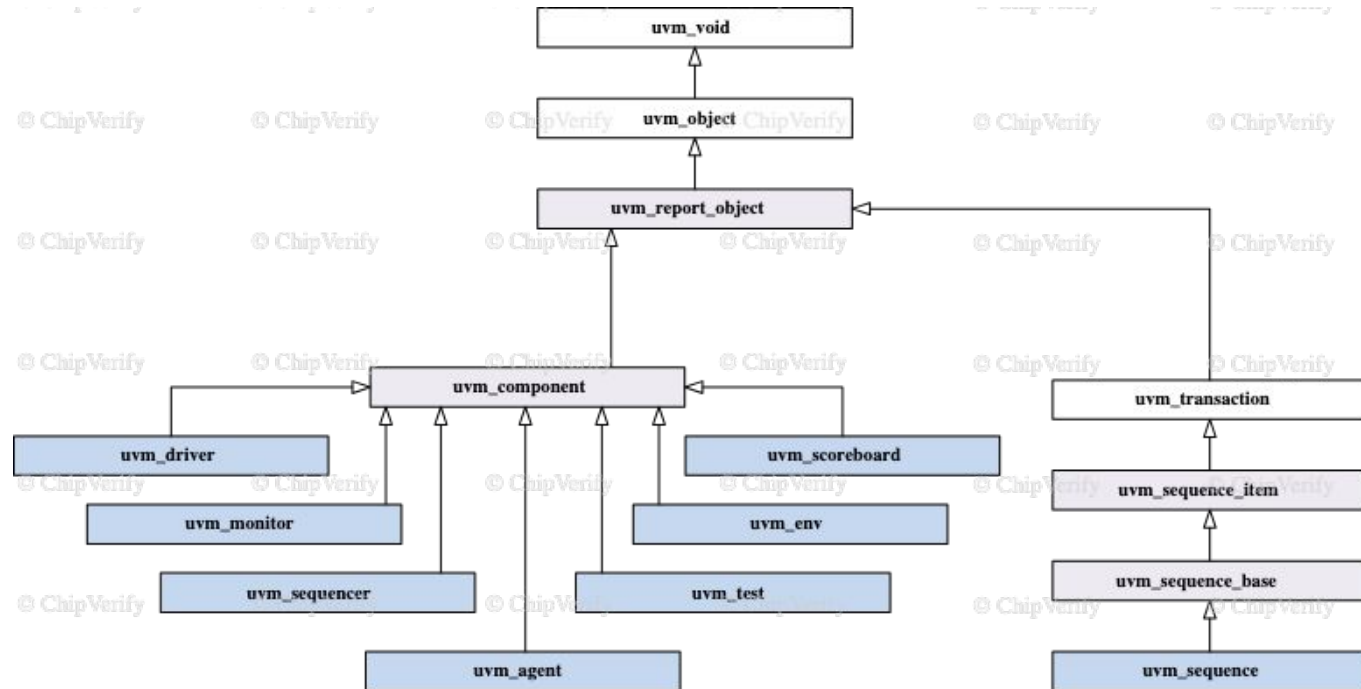
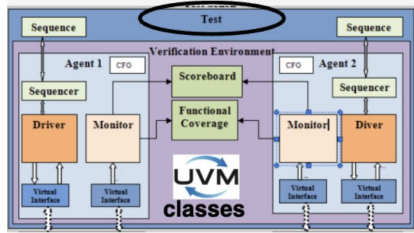
//-----
// Group -- NODOCS -- Simulation Control
//-----

// Task -- NODOCS -- run_test
//
// Phases all components through all registered phases. If the optional
// test_name argument is provided, or if a command-line plusarg,
// ++UVM_TESTNAME=TEST_NAME, is found, then the specified component is created
// just prior to phasing. The test may contain new verification components or
// the entire testbench, in which case the test and testbench can be chosen from
// the command line without forcing recompilation. If the global (package)
// variable, finish_on_completion, is set, then $finish is called after
// phasing completes.

extern virtual task run_test (string test_name="");
```

How UVM?

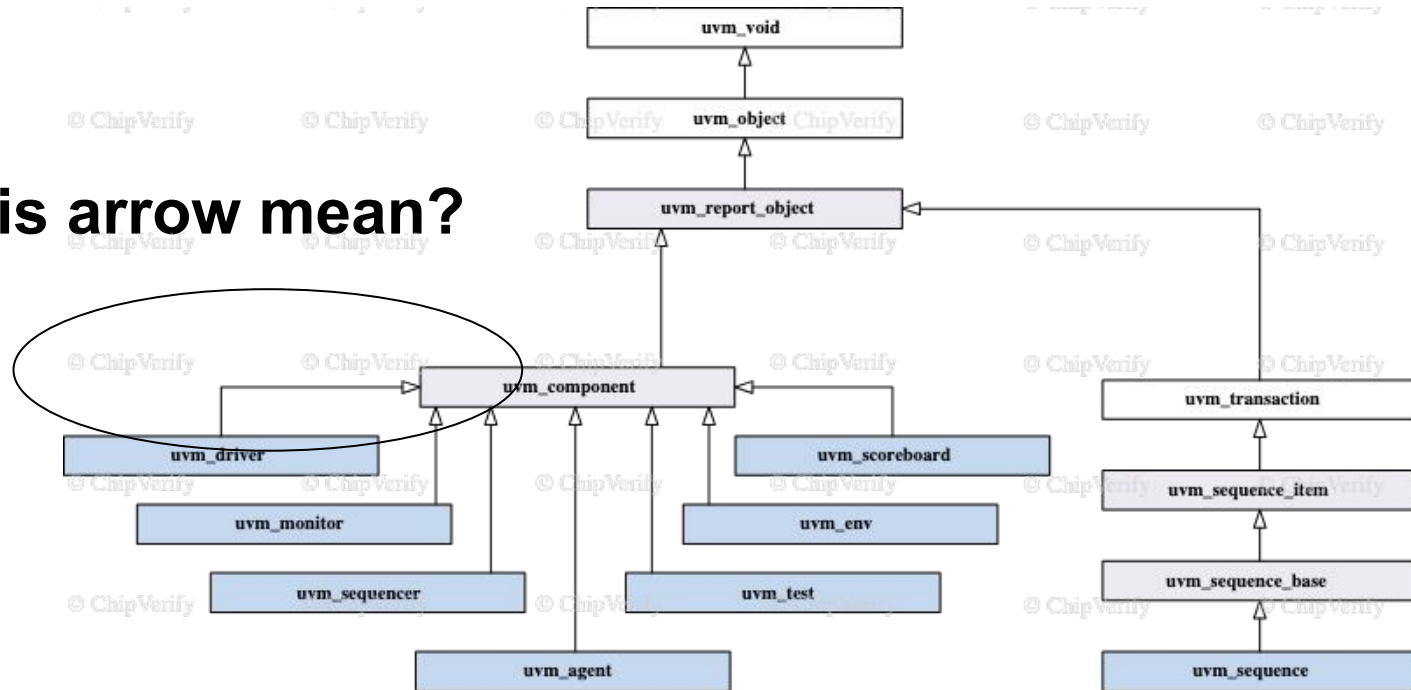
- These are the main classes;
- Just **derive** and **customize** it (remember this is a method)



How UVM?

- How? We need to understand the concept of POO

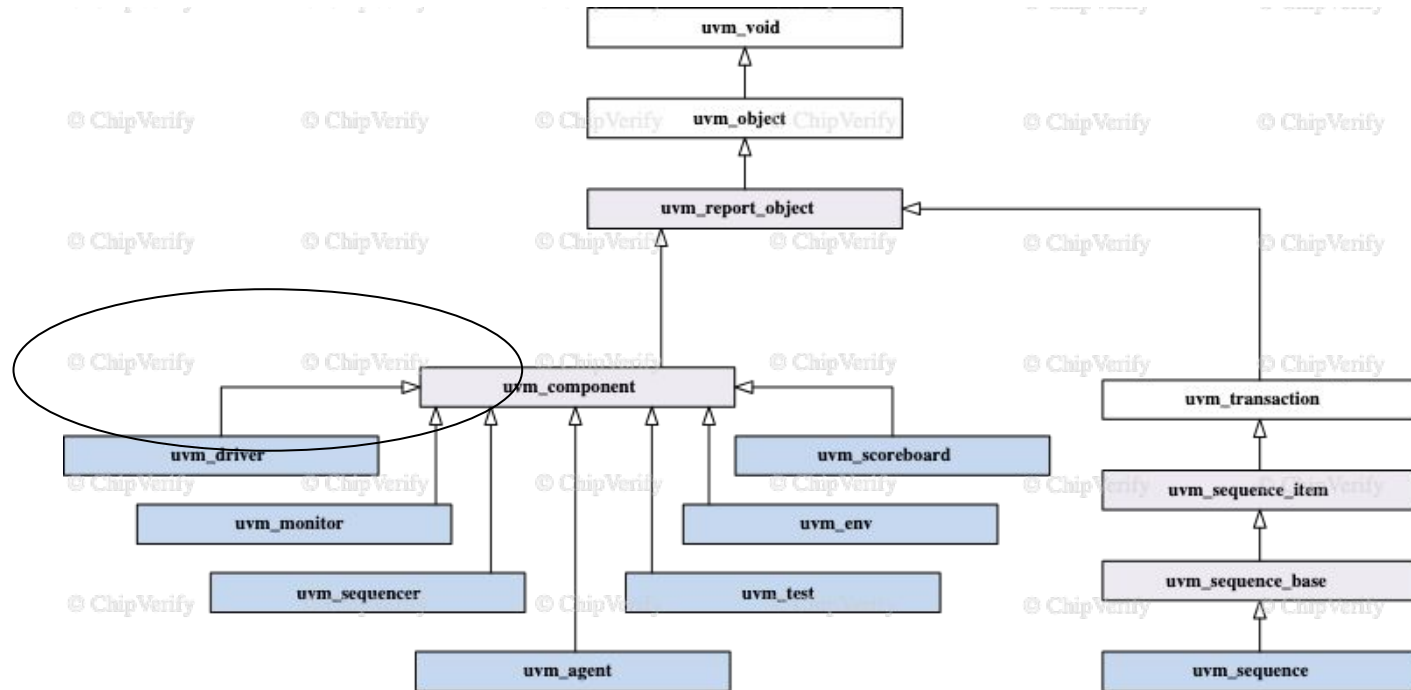
What does this arrow mean?



How UVM?

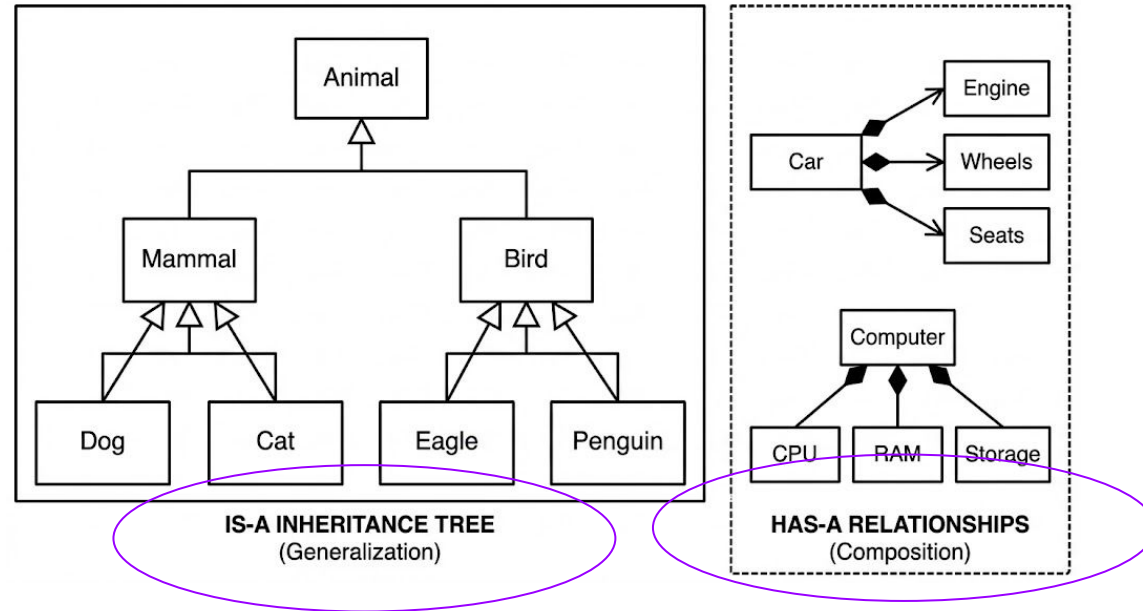
- How? We need to understand the concept of POO

- IS-A
- HAS-A



How UVM?

- IS-A
- HAS-A



How UVM?

O **polimorfismo** em orientação a objetos é a capacidade de um único método ou nome de função executar ações diferentes com base no objeto que o chama. Ele permite tratar objetos de classes diferentes de forma igual se eles compartilham uma **mesma base** ou **interface**.

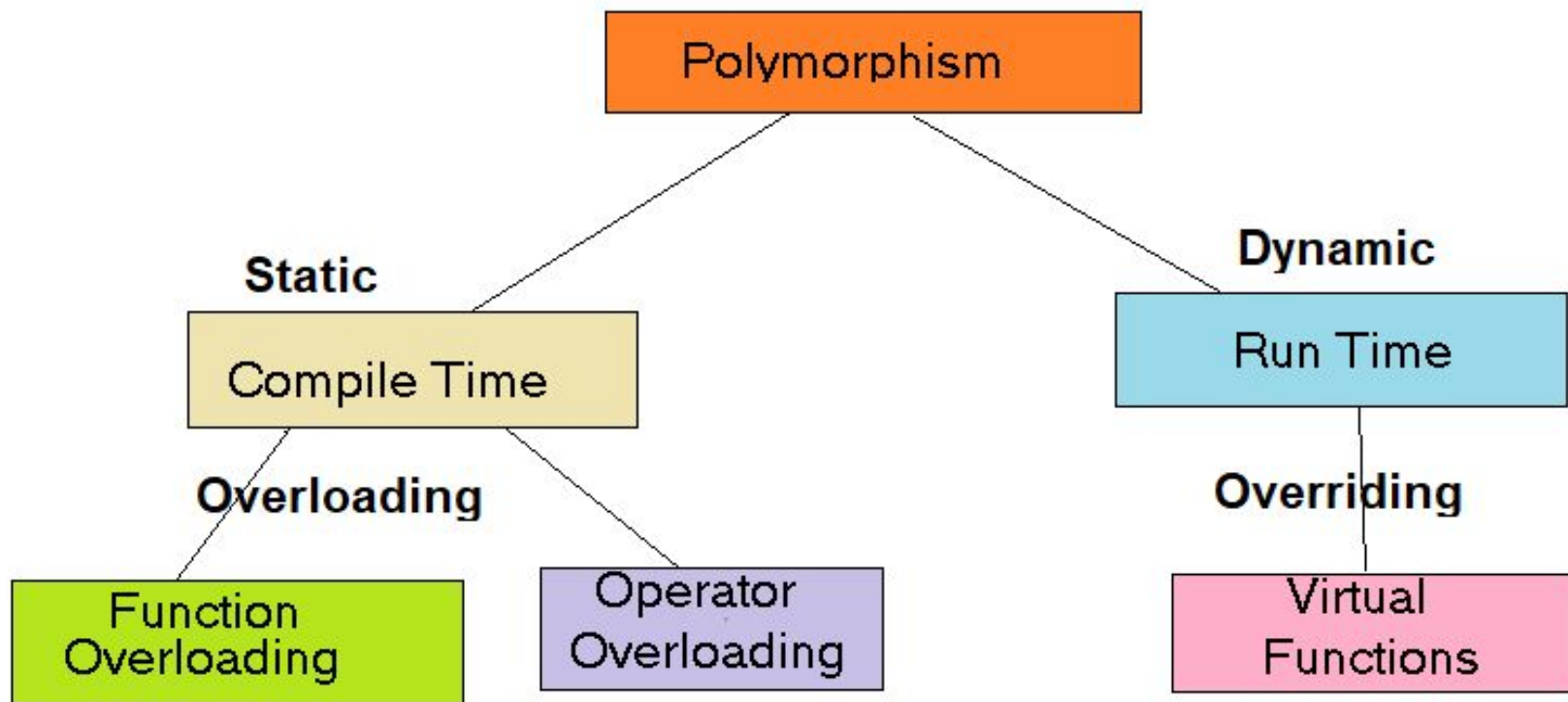
Relação com Outros Conceitos

- **Herança**: A base para o polimorfismo de inclusão. A subclasse herda da superclasse e muda o jeito que um método funciona.
- **Interfaces e Classes Abstratas**: Permitem criar regras comuns para várias classes sem usar herança direta.
- **Encapsulamento**: Esconde os detalhes do objeto. O programa só manda executar a ação sem se preocupar de qual forma ela acontece.

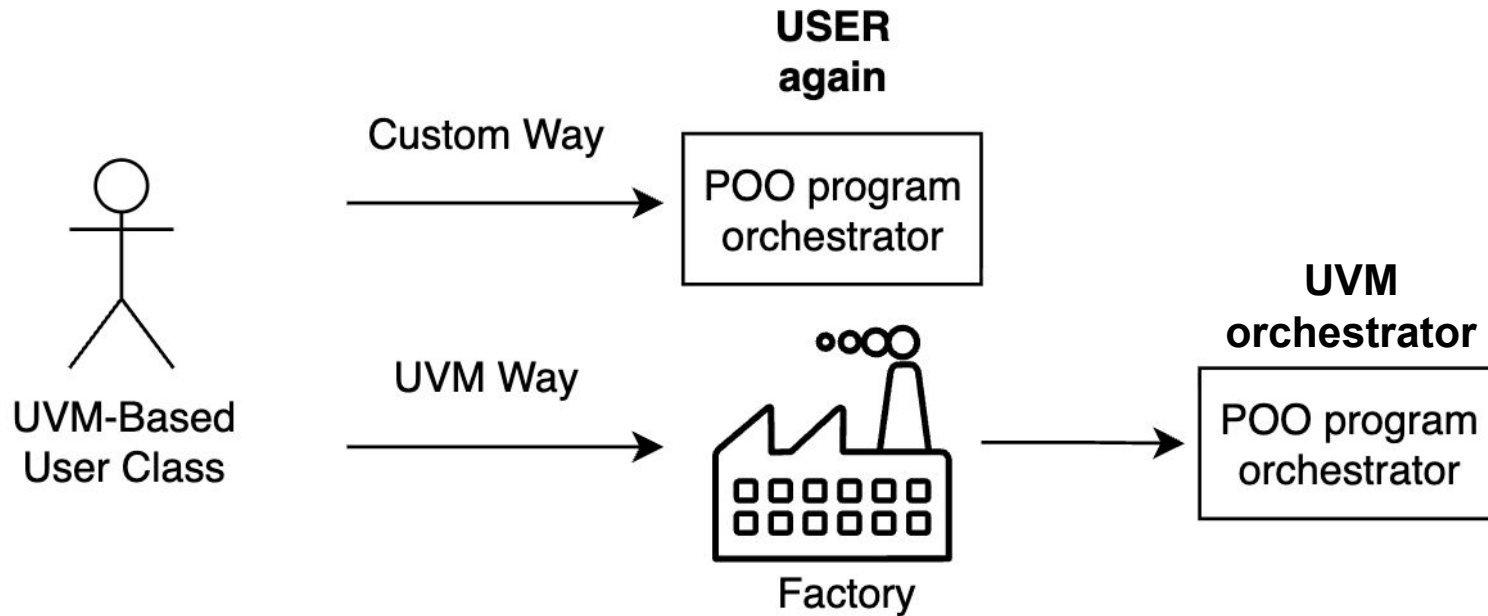
Tipos Principais

- **Sobrescrita (Override)**: Ocorre em tempo de execução. A subclasse refaz um método que veio da classe pai.
- **Sobrecarga (Overload)**: Ocorre em tempo de compilação. Vários métodos têm o mesmo nome na mesma classe, mas mudam os parâmetros

How UVM?



How UVM?

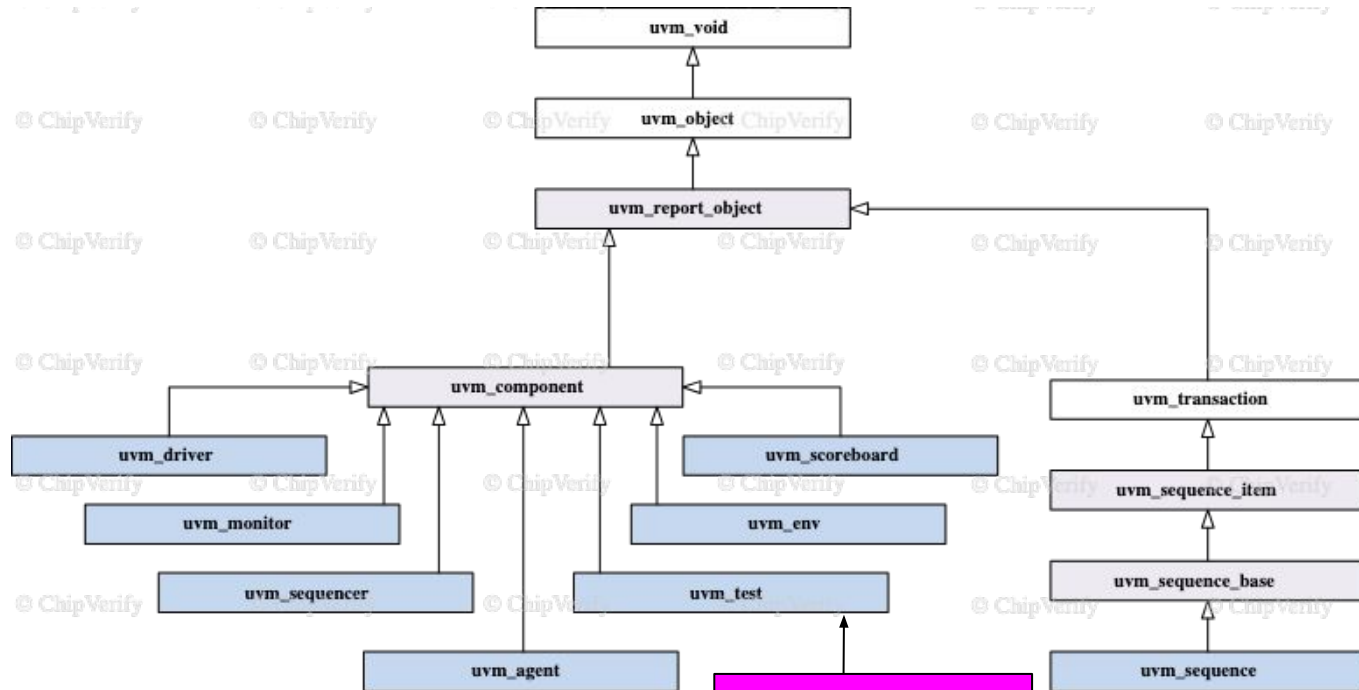


"The user is providing what they want in this part of the program..." This is a factory design pattern.

How UVM?

- Just **derive** and **customize** it (but remember your last GOAL)

- IS-A
- HAS-A



Our test IS-A
`uvm_test`

1800.2-2020.3.1

EXPLORER

src > comps > uvm_test.svh > ...

```
61 // If the specified test_name cannot be created by the <uvm_factory>, the
62 // fatal error occurs. If run_test() is called without UVM_TESTNAME being
63 // specified, then all components constructed before the call to run_test
64 // be cycled through their simulation phases.
65 //
66 // Deriving from uvm_test will allow you to distinguish tests from other
67 // component types that inherit from uvm_component directly. Such tests will
68 // automatically inherit features that may be added to uvm_test in the future
69 //
70 //-----
71
72 // @uvm-ieee 1800.2-2020 auto 13.2.1
73 virtual class uvm_test extends uvm_component;
74
75     `uvm_component_abstract_utils(uvm_test)
76
77     // Function -- NODOCS -- new
78     //
79     // Creates and initializes an instance of this class using the normal
80     // constructor arguments for <uvm_component>: ~name~ is the name of the
81     // instance, and ~parent~ is the handle to the hierarchical parent, if any.
82
83     // @uvm-ieee 1800.2-2020 auto 13.2.2
84     function new (string name, uvm_component parent);
85         super.new(name,parent);
86     endfunction
87
88 endclass
89
```

expands to

uvm_object_defines.svh

```
434 `uvm_field_utils_begin(T)
435 //
436 // @uvm-ieee 1800.2-2020 manual B.2.1.3
437 `define uvm_component_abstract_utils(T) \
438     `m_uvm_component_abstract_registry_internal(T,T) \
439     `uvm_type_name_decl("`T") \
440 //
441 // @uvm-ieee 1800.2-2020 manual B.2.1.3
442
```

expands to

uvm_object_defines.svh

```
634 //
635 // This is needed due to an issue in of passing down strings
636 // created by args to lower level macros.
637 `define m_uvm_component_abstract_registry_internal(T,S) \
638     typedef uvm_abstract_component_registry #(T,"S") type_id; \
639     static function type_id get_type(); \
640         return type_id::get(); \
641     endfunction \
642     virtual function uvm_object_wrapper get_object_type(); \
643         return type_id::get(); \
644     endfunction
645
646 // versions of the uvm_component_abstract_registry macros to be used wit
647 // parameterized classes
648
649 // m_uvm_component_abstract_registry_param
650
```

expands to

uvm_object_defines.svh

```
535 //
536 // This macro creates a statically accessible
537 // ~type_name~, and implements the virtual
538 // <uvm_object::get_type_name> method.
539 //
540 `define uvm_type_name_decl(TNAME_STRING) \
541     static function string type_name(); \
542         return TNAME_STRING; \
543     endfunction : type_name \
544     virtual function string get_type_name(); \
545         return TNAME_STRING; \
546     endfunction : get_type_name
547
548 // m_uvm_object_registry_internal
549 // -----
550
```

Launchpad 0 SystemVerilog: 387 indexed objects

Ln 543, Col 31 Spaces: 2 UTF-8 LF SystemVerilog

How UVM?

```
uvm_registry.svh
src > base -jGithub/svlog-design/svlog-uvm-libs/1800.2-2020.3/1src/base/uvm_registry.svh
307 // Class: uvm_abstract_component_registry#(T,Tname)
308 // Implementation of uvm_abstract_component_registry#(T,Tname), as defined by section
309 // 8.2.5.1.1 of 1800.2-2020.
310
311 // @uvm-ieee 1800.2-2020 auto 8.2.5.1.1
312 class uvm_abstract_component_registry #(type T=uvm_component, string Tname="<unknown>")
313     extends uvm_object_wrapper;
314     typedef uvm_abstract_component_registry #(T,Tname) this_type;
315     typedef uvm_registry_common#( this_type, uvm_registry_component_creator, T, Tname ) common_type;
316
317     // Function -- NODOCS -- create_component
318     //
319     // Creates a component of type T having the provided ~name~ and ~parent~.
320     // This is an override of the method in <uvm_object_wrapper>. It is
321     // called by the factory after determining the type of object to create.
322     // You should not call this method directly. Call <create> instead.
323
324     // @uvm-ieee 1800.2-2020 auto 8.2.5.1.2
325     virtual function uvm_component create_component (string name,
326                                                     uvm_component parent);
327     `uvm_error(
328         "UVM/ABST_RGTRY/CREATE_ABSTRACT_CMPNT",
329         $formatf( "Cannot create an instance of abstract class %s (with name %s and parent %s). Check for missing factor
330     )
331     return null;
332 endfunction
333
334 static function string type_name();
335     return common_type::type_name();
336 endfunction : type_name
337
338 // Function -- NODOCS -- get_type_name
339 //
340 // Returns the value given by the string parameter, ~Tname~. This method
341 // overrides the method in <uvm_object_wrapper>.
342
343 virtual function string get_type_name();
344     common_type common = common_type::get();
345     return common.get_type_name();
346 endfunction
```

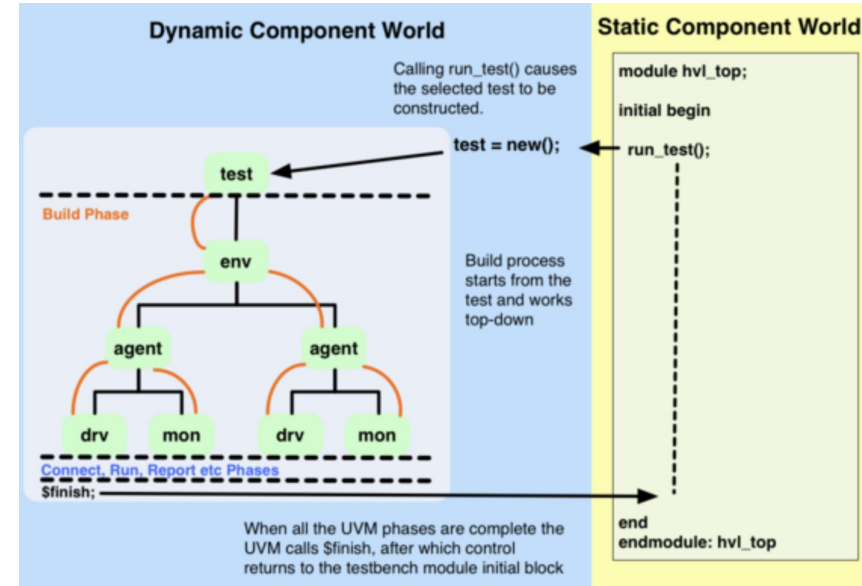
How UVM?

- POO behind it:

The `run_test()` is the entry point of a POO program.

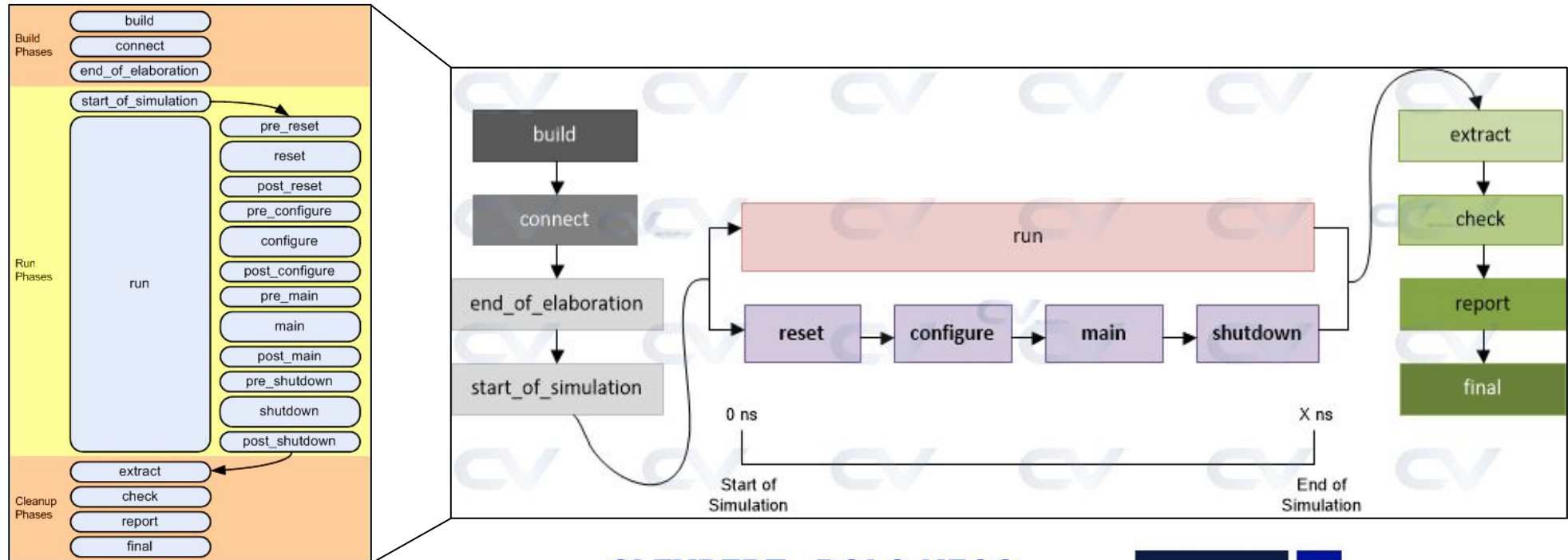
The POO program:

1. call `run_test()`;
2. the test starts a sequence;
3. the sequence generate items that are consumed by agents/seqr/drivers ;
4. driver drives interfaces;
5. interfaces are monitored;
6. others components use monitored data to test/check/analyze results.



How UVM?

- Each **UVM Component** has phases to be OVERRIDE/OVERLOAD
- Just **derive** and **customize** it (but remember your last GOAL)

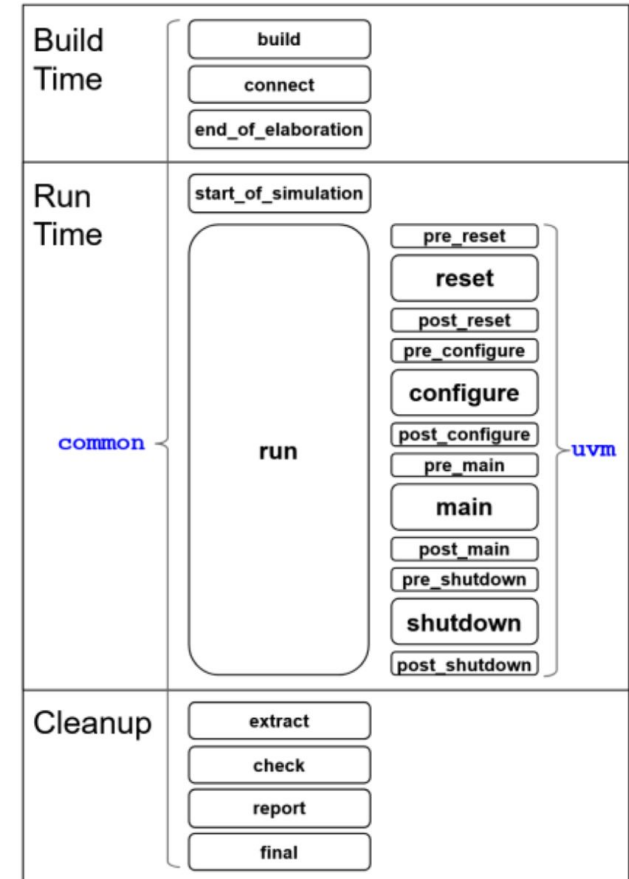


How UVM?

- But... what about **objections**?

- **common** - Environment components (environment/agent/driver/monitor)
 - ◆ run
- **uvm** - Sequence execution control (test only)
 - ◆ reset -> shutdown
 - With **pre_*/post_*** phases

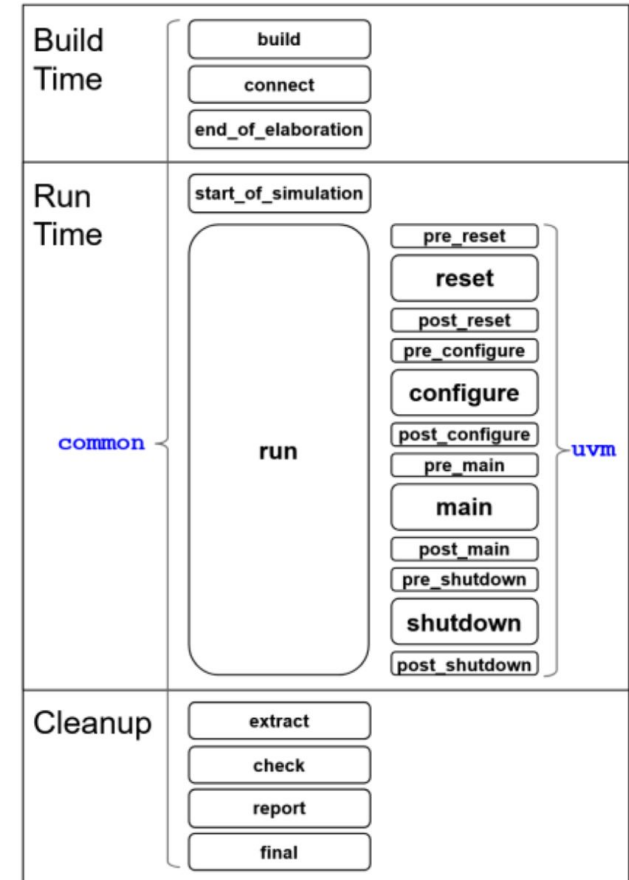
Task phases terminate when objection count reaches zero



How UVM?

- Example:

```
task my_test::run_phase(uvm_phase phase);  
    my_virtual_sequence vseq;  
  
    vseq = my_virtual_sequence::type_id::create("vseq");  
  
    phase.raise_objection(this, "Test scenario is active");  
  
    // Generate and send all input transactions  
    vseq.start(env.virtual_sequencer);  
  
    // Wait until all expected DUT outputs have been checked  
    env.scoreboard.wait_for_completion();  
  
    phase.drop_objection(this, "All expected outputs were checked");  
endtask
```

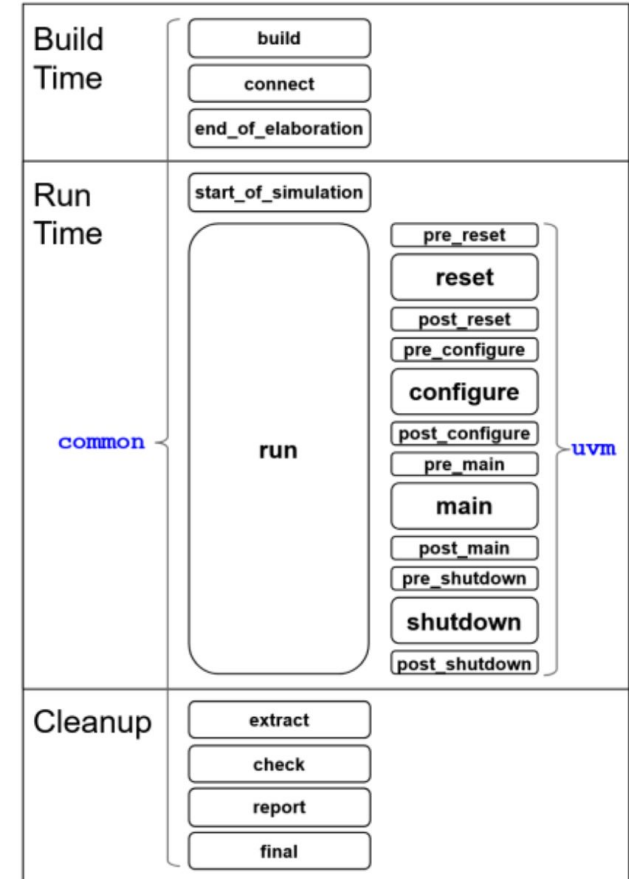


How UVM?

- Example:

The objection does not cause the task to execute; it makes the UVM wait for the work being performed.

```
task reset_phase(uvm_phase phase);  
    phase.raise_objection(this, "Applying reset");  
  
    vif.rst_n <= 1'b0;  
    repeat (10) @(posedge vif.clk);  
    vif.rst_n <= 1'b1;  
  
    phase.drop_objection(this, "Reset completed");  
endtask
```

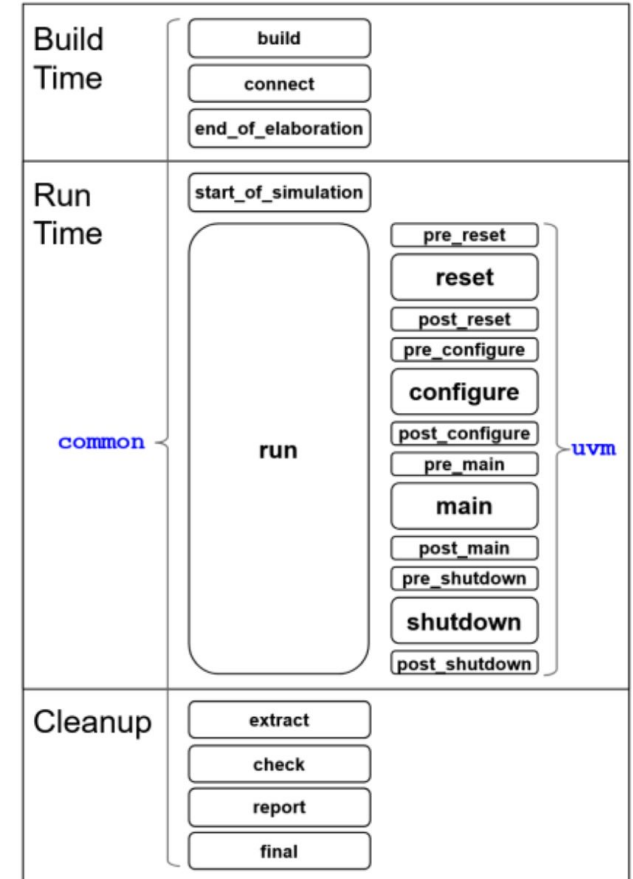


How UVM?

- Example:

Why run_phase does not have objection? Because it never finishes...

```
task run_phase(uvm_phase phase);  
  
    forever begin  
        @(posedge vif.clk);  
        monitor();  
    end  
  
endtask
```



LAB00:

"Hello, World!"

from test class



LAB00: "Hello, World" from test class

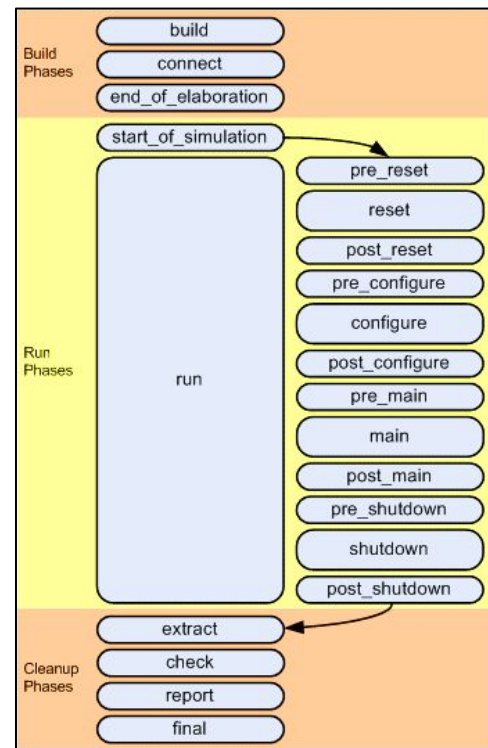
- **GOAL:**
 - Understand how test is called and how to modify/override methods
- **1) Run \$ *vcs_fall* of LAB00**

module: blockname_top_tb

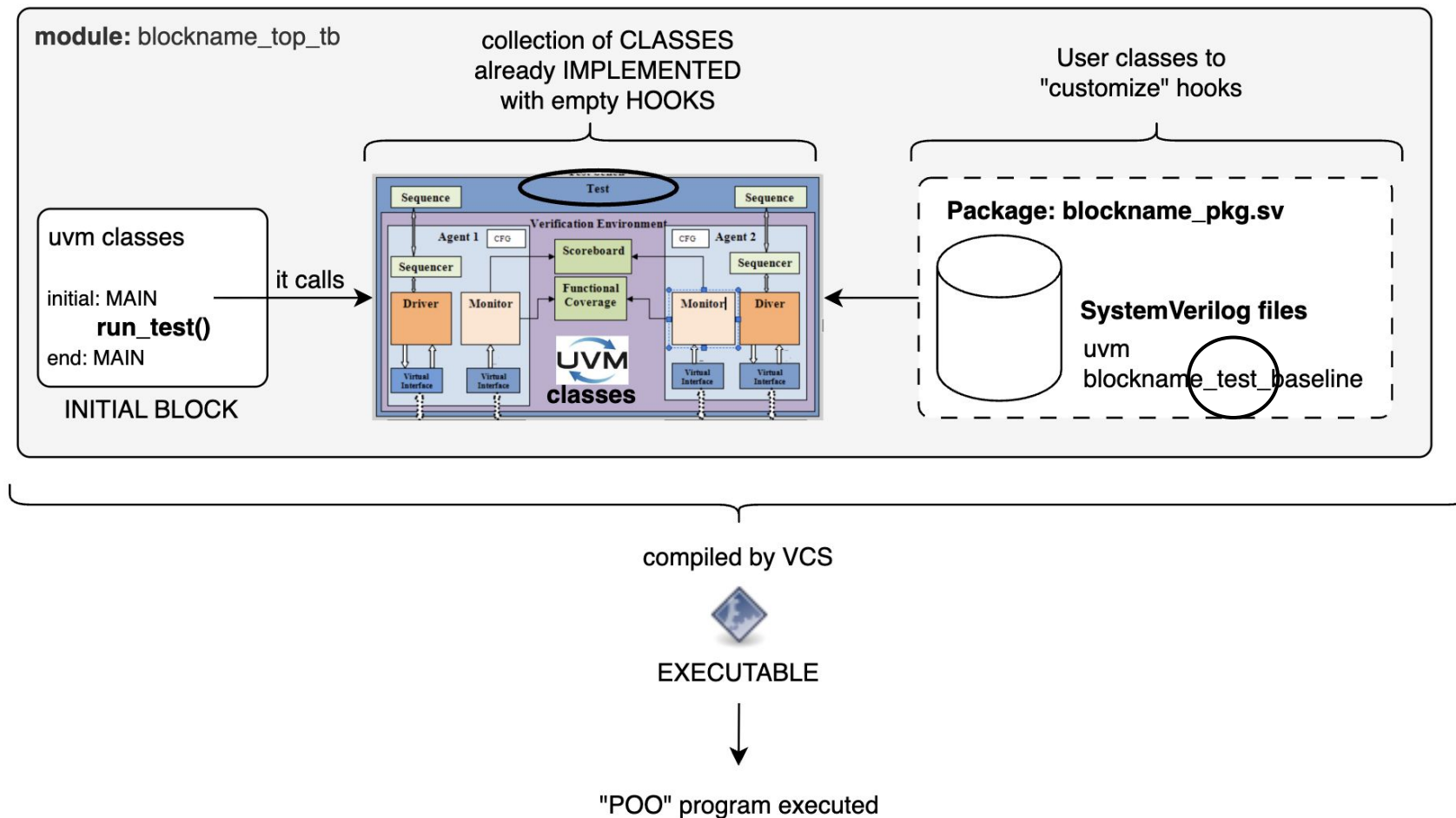
initial: MAIN

LAB00: "Hello, World" from test class

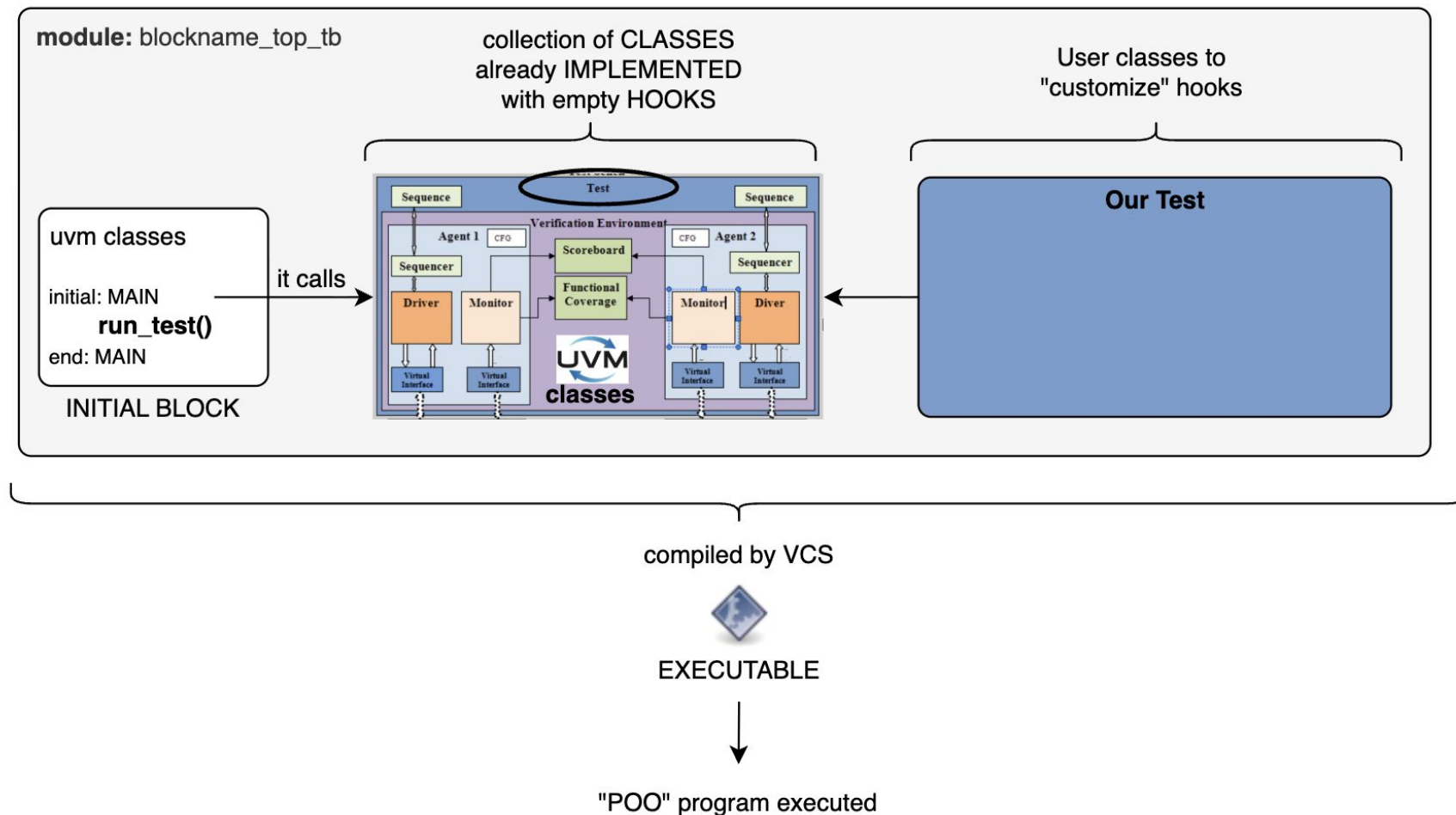
- **GOAL:**
 - Understand how test is called and how to modify/override methods
- **1) Run `$ vcs_fall` of LAB00**
- **2) Adds a reference/base/baseline test**



LAB00: "Hello, World" from test class

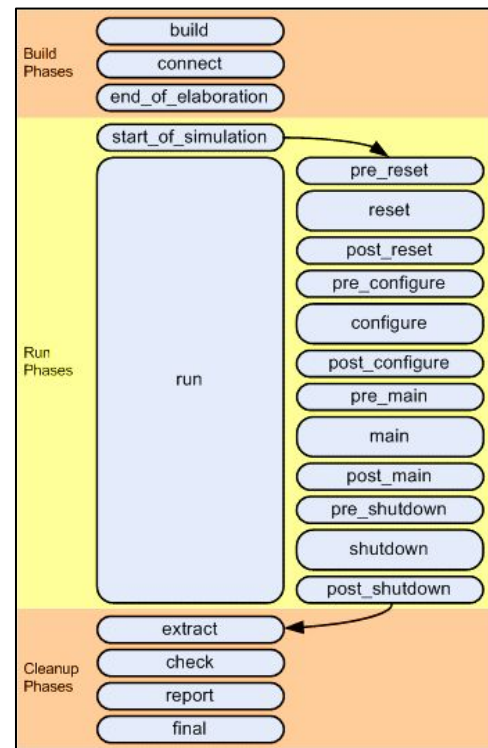


LAB00: "Hello, World" from test class

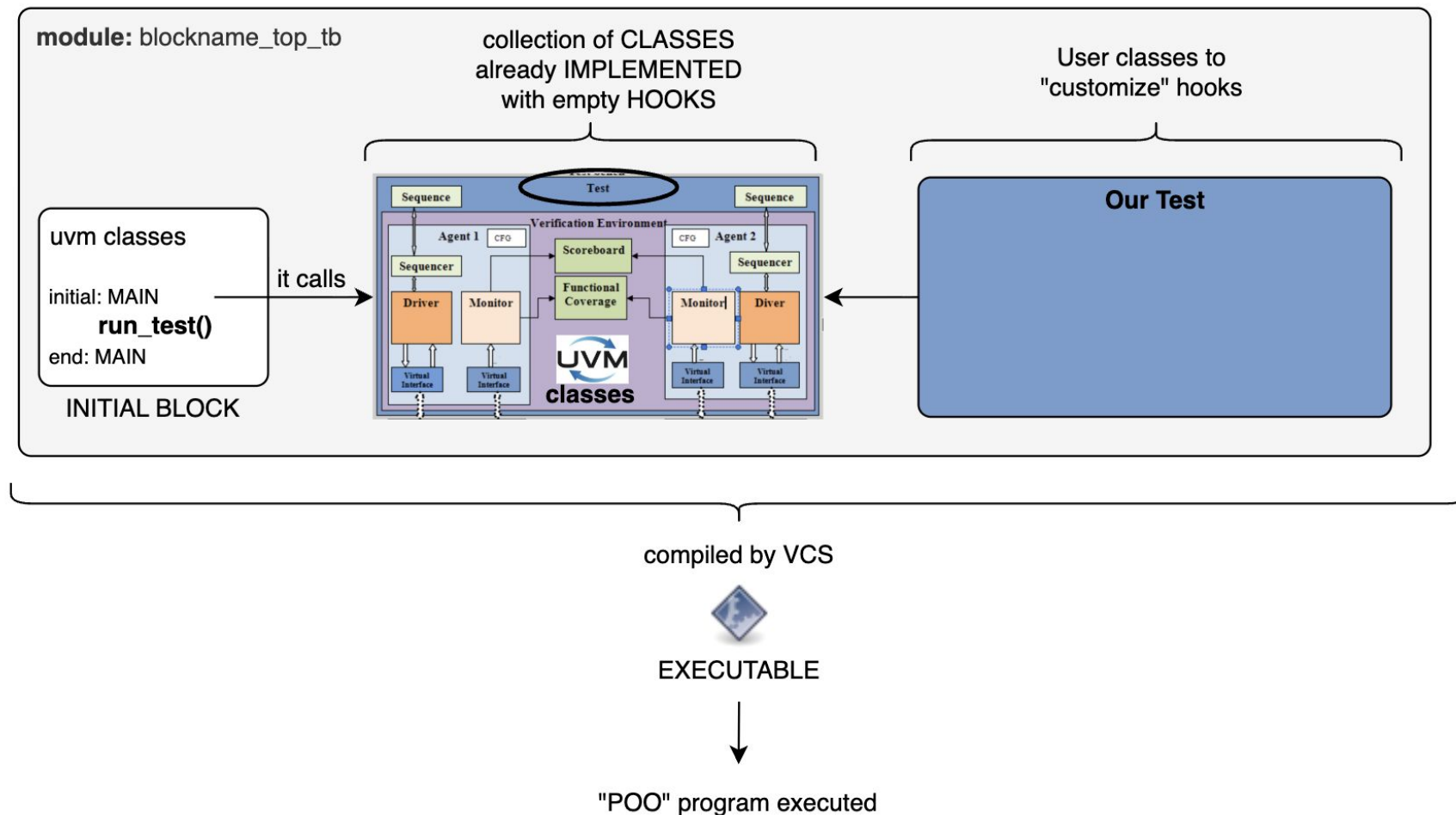


LAB00: "Hello, World" from test class

- **GOAL:**
 - Understand how test is called and how to modify/override methods
- 3) Get factory error
- 4) Fix factory error
- 5) Override functions with uvm_info
- 6) Test #1ms in some phases

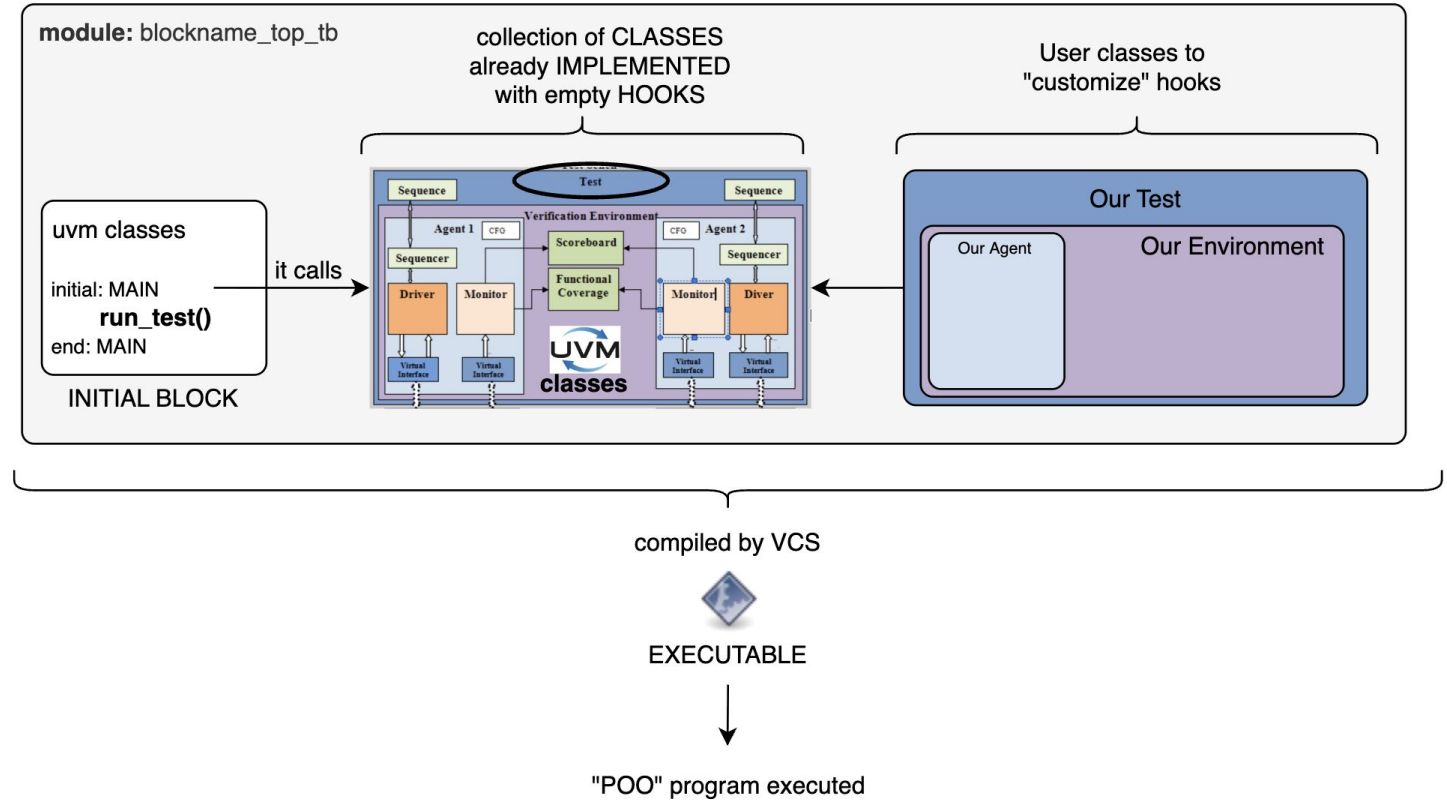


LAB00: "Hello, World" from test class



EXERCISE

Criar os componentes do test e sobrescrever metodos/hooks para provar a execução na especificação.





+Components

next lesson...

RESIDÊNCIA EM MICROELETRÔNICA - POLO UFMG



TKS

EXECUÇÃO:



APOIO:



INICIATIVA:

